

# Thrust Vectoring

## *Component Design Document*

## 1 Description

Thrust vectoring. Computes the unit direction of a fixed magnitude thrust acting through a fixed point on the vehicle so that it produces the requested torque about the center of mass. The part of the request along the moment arm cannot be produced and is discarded, and a request beyond what the geometry can deliver saturates at the largest achievable torque. Wraps the ThrustVectoringAlgorithm C++ algorithm via its C shim.

## 2 Requirements

No requirements have been specified for this component.

## 3 Design

### 3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 4
- **Number of Invokee Connectors** - 2
- **Number of Invoker Connectors** - 2
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - 3
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - 1
- **Number of Data Dependencies** - 1
- **Number of Packets** - *None*

## 3.2 Diagram

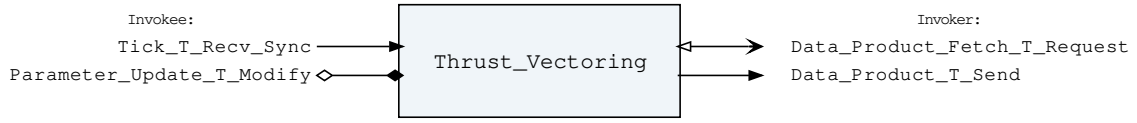


Figure 1: Thrust Vectoring component diagram.

## 3.3 Connectors

Below are tables listing the component's connectors.

### 3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Thrust Vectoring Invokee Connectors

| Name                      | Kind      | Type               | Return_Type | Count |
|---------------------------|-----------|--------------------|-------------|-------|
| Tick_T_Recv_Sync          | recv_sync | Tick.T             | -           | 1     |
| Parameter_Update_T_Modify | modify    | Parameter_Update.T | -           | 1     |

Connector Descriptions:

- **Tick\_T\_Recv\_Sync** - Run the algorithm up to the current time.
- **Parameter\_Update\_T\_Modify** - The parameter update connector.

### 3.3.2 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 2: Thrust Vectoring Invoker Connectors

| Name                         | Kind    | Type                 | Return_Type           | Count |
|------------------------------|---------|----------------------|-----------------------|-------|
| Data_Product_Fetch_T_Request | request | Data_Product_Fetch.T | Data_Product_Return.T | 1     |
| Data_Product_T_Send          | send    | Data_Product.T       | -                     | 1     |

Connector Descriptions:

- **Data\_Product\_Fetch\_T\_Request** - Fetch a data product item from the database.
- **Data\_Product\_T\_Send** - The data product invoker connector

## 3.4 Interrupts

This component contains no interrupts.

## 3.5 Initialization

Below are details on how the component should be initialized in an assembly.

### 3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

### 3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

### 3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 3: Thrust Vectoring Set Id Bases Parameters

| Name                              | Type                                                 |
|-----------------------------------|------------------------------------------------------|
| <code>Data_Product_Id_Base</code> | <code>Data_Product_Types.Data_Product_Id_Base</code> |
| <code>Parameter_Id_Base</code>    | <code>Parameter_Types.Parameter_Id_Base</code>       |

Parameter Descriptions:

- **Data\_Product\_Id\_Base** - The value at which the component's data product identifiers begin.
- **Parameter\_Id\_Base** - The value at which the component's parameter identifiers begin.

### 3.5.4 Component Map Data Dependencies

This component contains data dependencies that need their IDs mapped to valid data product IDs at initialization. Each data dependency also needs a stale limit provided in microseconds. If the timestamp of the data dependency is older than the reference time (usually the current time) minus the stale limit then the data dependency is considered too old to safely use. The `map_Data_Dependencies` procedure must be called with the following parameters:

Table 4: Thrust Vectoring Map Data Dependencies Parameters

| Name                                    | Type                                            |
|-----------------------------------------|-------------------------------------------------|
| <code>Torque_Request_Id</code>          | <code>Data_Product_Types.Data_Product_Id</code> |
| <code>Torque_Request_Stale_Limit</code> | <code>Ada.Real_Time.Time_Span</code>            |

Parameter Descriptions:

- **Torque\_Request\_Id** - The external data product ID used to fetch the value for the 'Torque\_Request' data dependency.
- **Torque\_Request\_Stale\_Limit** - The stale limit used for the 'Torque\_Request' data dependency. If the fetched data product is older than the given `Time_Span` then it is deemed stale and unfit for use by the component.

### 3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the thrust vectoring algorithm with the default parameter values. The `init` subprogram takes no parameters.

## 3.6 Commands

The Thrust Vectoring component has no commands.

### 3.7 Parameters

Parameters for the Thrust Vectoring component.

Table 5: Thrust Vectoring Parameters

| Local ID   | Parameter Name | Type           | Default Value   |
|------------|----------------|----------------|-----------------|
| 0x0000 (0) | R_Mb_B         | Packed_F32x3.T | [0.0, 0.0, 1.0] |
| 0x0001 (1) | Thrust         | Packed_F32.T   | (Value=>1.0)    |
| 0x0002 (2) | R_Cb_B         | Packed_F32x3.T | [0.0, 0.0, 0.0] |

Parameter Descriptions:

- **R\_Mb\_B** - [m] Position of the thrust point M relative to the body frame origin, body frame components. Must be finite.
- **Thrust** - [N] Thrust magnitude. Must be finite and greater than zero.
- **R\_Cb\_B** - [m] Position of the center of mass relative to the body frame origin, body frame components. Must be finite and more than 1 mm from the thrust point.

### 3.8 Events

The Thrust Vectoring component has no events.

### 3.9 Data Products

Data products for the Thrust Vectoring component.

Table 6: Thrust Vectoring Data Products

| Local ID   | Data Product Name | Type           |
|------------|-------------------|----------------|
| 0x0000 (0) | Thrust_Direction  | Packed_F32x3.T |

Data Product Descriptions:

- **Thrust\_Direction** - [-] Unit thrust direction in body frame components that produces the requested torque. A zero request fires the thrust through the center of mass.

### 3.10 Data Dependencies

Data dependencies for the Thrust Vectoring component.

Table 7: Thrust Vectoring Data Dependencies

| Data Dependency Name | Type              |
|----------------------|-------------------|
| Torque_Request       | Cmd_Torque_Body.T |

Data Dependency Descriptions:

- **Torque\_Request** - [Nm] Requested thruster torque about the center of mass, body frame components.

### 3.11 Packets

The Thrust Vectoring component has no packets.

### 3.12 Faults

The Thrust Vectoring component has no faults.

## 4 Unit Tests

The following section describes the unit test suites written to test the component.

### 4.1 *Thrust\_Vectoring\_Tests* Test Suite

This is a unit test suite for the Thrust Vectoring component

Test Descriptions:

- **Test** - Point the thrust for a zero, a small, and a saturating torque request to ensure the Ada to C to C++ integration is sound.
- **Test\_Invalid\_Parameter** - Ensure a staged configuration the algorithm would reject is refused at validation.

## 5 Appendix

### 5.1 Preamble

This component contains no preamble code.

### 5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

#### Cmd\_Torque\_Body.T:

Commanded control torque in body frame components, as produced by the MRP control laws.

Table 8: Cmd\_Torque\_Body Packed Record : 96 bits

| Name                | Type           | Range | Size (Bits) | Start Bit | End Bit |
|---------------------|----------------|-------|-------------|-----------|---------|
| Torque_Request_Body | Packed_F32x3.T | -     | 96          | 0         | 95      |

Field Descriptions:

- **Torque\_Request\_Body** - [Nm] Control torque requested

#### Data\_Product.T:

Generic data product packet for holding arbitrary data types

Table 9: Data\_Product Packed Record : 1096 bits (*maximum*)

| Name   | Type                                        | Range | Size (Bits) | Start Bit | End Bit | Variable Length      |
|--------|---------------------------------------------|-------|-------------|-----------|---------|----------------------|
| Header | Data_Product_Header.T                       | -     | 72          | 0         | 71      | -                    |
| Buffer | Data_Product_Types.Data_Product_Buffer_Type | -     | 1024        | 72        | 1095    | Header.Buffer_Length |

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

### Data\_Product\_Fetch.T:

A packed record which holds information for a data product request.

Table 10: Data\_Product\_Fetch Packed Record : 16 bits

| Name | Type                               | Range      | Size (Bits) | Start Bit | End Bit |
|------|------------------------------------|------------|-------------|-----------|---------|
| Id   | Data_Product_Types.Data_Product_Id | 0 to 65535 | 16          | 0         | 15      |

Field Descriptions:

- **Id** - The data product identifier

### Data\_Product\_Header.T:

Generic data\_product packet for holding arbitrary data\_product types

Table 11: Data\_Product\_Header Packed Record : 72 bits

| Name          | Type                                               | Range      | Size (Bits) | Start Bit | End Bit |
|---------------|----------------------------------------------------|------------|-------------|-----------|---------|
| Time          | Sys_Time.T                                         | -          | 48          | 0         | 47      |
| Id            | Data_Product_Types.Data_Product_Id                 | 0 to 65535 | 16          | 48        | 63      |
| Buffer_Length | Data_Product_Types.Data_Product_Buffer_Length_Type | 0 to 128   | 8           | 64        | 71      |

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer\_Length** - The number of bytes used in the data product buffer

### Data\_Product\_Return.T:

This record holds data returned from a data product fetch request.

Table 12: Data\_Product\_Return Packed Record : 1104 bits (*maximum*)

| Name                     | Type                                              | Range                                                      | Size (Bits) | Start Bit | End Bit | Variable Length |
|--------------------------|---------------------------------------------------|------------------------------------------------------------|-------------|-----------|---------|-----------------|
| The_<br>Status           | Data_<br>Product_<br>Enums.<br>Fetch_<br>Status.E | 0 => Success<br>1 => Not_Available<br>2 => Id_Out_Of_Range | 8           | 0         | 7       | -               |
| The_<br>Data_<br>Product | Data_<br>Product.<br>T                            | -                                                          | 1096        | 8         | 1103    | -               |

Field Descriptions:

- **The\_Status** - A status relating whether or not the data product fetch was successful or not.
- **The\_Data\_Product** - The data product item returned.

### Packed\_F32.T:

Single component record for holding packed 32-bit floating point number.

Table 13: Packed\_F32 Packed Record : 32 bits

| Name  | Type        | Range                       | Size (Bits) | Start Bit | End Bit |
|-------|-------------|-----------------------------|-------------|-----------|---------|
| Value | Short_Float | -3.40282e+38 to 3.40282e+38 | 32          | 0         | 31      |

Field Descriptions:

- **Value** - The 32-bit floating point number.

### Packed\_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 14: Packed\_F32x3 Packed Array : 96 bits

| Type               | Range                       | Element Size (Bits) | Length | Total Size (Bits) |
|--------------------|-----------------------------|---------------------|--------|-------------------|
| <b>Short_Float</b> | -3.40282e+38 to 3.40282e+38 | 32                  | 3      | 96                |

### Parameter.T:

Generic parameter packet for holding a generic parameter

Table 15: Parameter Packed Record : 792 bits (*maximum*)

| Name   | Type                                              | Range | Size (Bits) | Start Bit | End Bit | Variable Length          |
|--------|---------------------------------------------------|-------|-------------|-----------|---------|--------------------------|
| Header | Parameter_<br>Header.T                            | -     | 24          | 0         | 23      | -                        |
| Buffer | Parameter_<br>Types.<br>Parameter_<br>Buffer_Type | -     | 768         | 24        | 791     | Header.Buffer_<br>Length |

---

Field Descriptions:

- **Header** - The parameter header
- **Buffer** - A buffer that contains the parameter type

### Parameter\_Header.T:

Generic parameter header for holding arbitrary parameters

Table 16: Parameter\_Header Packed Record : 24 bits

| Name          | Type                                                 | Range      | Size (Bits) | Start Bit | End Bit |
|---------------|------------------------------------------------------|------------|-------------|-----------|---------|
| Id            | Parameter_Types.<br>Parameter_Id                     | 0 to 65535 | 16          | 0         | 15      |
| Buffer_Length | Parameter_Types.<br>Parameter_Buffer_<br>Length_Type | 0 to 96    | 8           | 16        | 23      |

Field Descriptions:

- **Id** - The parameter identifier
- **Buffer\_Length** - The number of bytes used in the parameter type buffer

### Parameter\_Update.T:

A record intended to be used as a provide/modify connector type for updating/fetching parameters.

Table 17: Parameter\_Update Packed Record : 824 bits (*maximum*)

| Name      | Type                                           | Range                                                                       | Size (Bits) | Start Bit | End Bit | Variable Length |
|-----------|------------------------------------------------|-----------------------------------------------------------------------------|-------------|-----------|---------|-----------------|
| Table_Id  | Parameter_Types.<br>Parameter_Table_Id         | 0 to 65535                                                                  | 16          | 0         | 15      | –               |
| Operation | Parameter_Enums.<br>Parameter_Operation_Type.E | 0 => Stage<br>1 => Update<br>2 => Fetch<br>3 => Validate                    | 8           | 16        | 23      | –               |
| Status    | Parameter_Enums.<br>Parameter_Update_Status.E  | 0 => Success<br>1 => Id_Error<br>2 => Validation_Error<br>3 => Length_Error | 8           | 24        | 31      | –               |
| Param     | Parameter.T                                    | –                                                                           | 792         | 32        | 823     | –               |

Field Descriptions:

- **Table\_Id** - The ID for the table that contains this parameter
- **Operation** - The parameter operation to perform.
- **Status** - The parameter return status.
- **Param** - The parameter that has been updated or fetched.

### Sys\_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 18: Sys\_Time Packed Record : 48 bits

| Name       | Type                       | Range           | Size (Bits) | Start Bit | End Bit |
|------------|----------------------------|-----------------|-------------|-----------|---------|
| Seconds    | Interfaces.<br>Unsigned_32 | 0 to 4294967295 | 32          | 0         | 31      |
| Subseconds | Interfaces.<br>Unsigned_16 | 0 to 65535      | 16          | 32        | 47      |

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of  $1/(2^{16})$  sub-seconds.

### Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 19: Tick Packed Record : 80 bits

| Name  | Type                       | Range           | Size (Bits) | Start Bit | End Bit |
|-------|----------------------------|-----------------|-------------|-----------|---------|
| Time  | Sys_Time.T                 | -               | 48          | 0         | 47      |
| Count | Interfaces.<br>Unsigned_32 | 0 to 4294967295 | 32          | 48        | 79      |

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

## 5.3 Enumerations

The following section outlines any enumerations used in the component.

### Data\_Product\_Enums.Fetch\_Status.E:

This status denotes whether a data product fetch was successful.

Table 20: Fetch\_Status Literals:

| Name    | Value | Description                                 |
|---------|-------|---------------------------------------------|
| Success | 0     | The data product was returned successfully. |

|                 |   |                                                       |
|-----------------|---|-------------------------------------------------------|
| Not_Available   | 1 | No data product is yet available for the provided id. |
| Id_Out_Of_Range | 2 | The data product id was out of range.                 |

### Parameter\_Enums.Parameter\_Operation\_Type.E:

This enumeration lists the different parameter operations that can be performed.

Table 21: Parameter\_Operation\_Type Literals:

| Name     | Value | Description                                                       |
|----------|-------|-------------------------------------------------------------------|
| Stage    | 0     | Stage the parameter.                                              |
| Update   | 1     | All parameters are staged, it is ok to update all parameters now. |
| Fetch    | 2     | Fetch the parameter.                                              |
| Validate | 3     | Validate all the parameters.                                      |

### Parameter\_Enums.Parameter\_Update\_Status.E:

This status enumeration provides information on the success/failure of a parameter operation.

Table 22: Parameter\_Update\_Status Literals:

| Name             | Value | Description                                       |
|------------------|-------|---------------------------------------------------|
| Success          | 0     | Parameter was successfully staged.                |
| Id_Error         | 1     | Parameter id was not valid.                       |
| Validation_Error | 2     | Parameter values were not successfully validated. |
| Length_Error     | 3     | Parameter length was not correct.                 |